Assignment 2: Retail Sales Summary Tool
Sprint Report

Course: CPSC-620-3 Agile Software Development — Winter 2026

Team Members: EUN SOO PARK, Sunghoon Ahn, Mohammad Shoaib, Sannibhai Indradeepbhai Garasiya

Sprint Duration: March 2 — March 8, 2026 (1 week)

GitHub Repository: https://github.com/nagisatosanae/retail-sales-summary-tool

Taiga Project: https://tree.taiga.io/project/espark-retail-sales-summary-tool/timeline

1. Team Liftoff Summary

At the beginning of the sprint, our team held a short liftoff meeting to align on the scope of the project, clarify expectations, and decide how we would collaborate during the one-week sprint. Since this was our first sprint together, it was important for us to understand each member's background and level of familiarity with the tools we would be using.

During the discussion, we shared our individual strengths. Some members were more comfortable working with Python and data analysis, while others had more experience with Git workflows or documentation. This helped us distribute responsibilities in a practical way and ensured that each member could contribute effectively throughout the sprint.

To guide our collaboration, we also created a simple team charter.

Mission & Vision:

Our mission was to build a modular Python tool that can summarize retail sales data and generate meaningful insights from the Superstore dataset. At the same time, we aimed to practice real Agile collaboration using GitHub and Taiga rather than simply completing the technical task individually.

Success Priorities:

We agreed on several priorities for success:

- Every team member must contribute through code, review, or documentation.
- The tool should produce correct and verifiable outputs.
- Agile artifacts such as user stories, backlog tracking, and commits should be traceable.

Working Agreements

The team agreed to communicate daily through Microsoft Teams. Questions related to Git, coding issues, or task coordination were discussed openly so that no one would remain blocked for long.

Definition of Done:

- Feature code is reviewed and merged via PR.
- Functions include docstrings and return documented data structures.
- Related tests pass (where applicable).

- The Taiga task is moved to "Closed" after the code is reviewed and merged.

2. Sprint Goal

The sprint goal was to deliver a functional Python tool that reads the Superstore dataset and produces meaningful summaries related to sales performance, profit analysis, customer behavior, time-based trends, and shipping patterns.

In addition to producing working code, the sprint also focused on demonstrating Agile practices such as backlog planning, traceable commits, feature branches, and task tracking using Taiga.

3. User Stories and Backlog

We defined 7 user stories, estimated with story points using Planning Poker, and prioritized them based on dependency order and business value.

ID	User Story	Story Points	Priority	Assigned To
US-01	As a data analyst, I want to load and clean the superstore dataset so that I can work with reliable, consistent data.	3	1 (Highest)	EUN SOO PARK
US-02	As a business manager, I want to see sales summaries by category and region so that I can understand revenue distribution.	5	2	EUN SOO PARK
US-03	As a financial analyst, I want to analyze profit margins across products and segments so that I can identify high and low performers.	5	3	Sunghoon Ahn
US-04	As a marketing manager, I want to understand customer purchasing patterns so that I can target the right segments.	3	4	Sannibhai Indradeepbhai Garasiya
US-05	As a sales director, I want to see sales trends over time so that I can identify seasonal patterns and growth trajectories.	5	5	EUN SOO PARK
US-06	As a developer, I want reusable visualization functions that return plot objects so that any dashboard tool can consume them.	3	6	Sunghoon Ahn
US-07	As an operations manager, I want to analyze shipping modes and delivery times so that I can optimize logistics decisions.	3	7	Mohammad Shoaib

Total sprint capacity was 27 story points

Each story was divided into smaller tasks inside Taiga and assigned to specific team members.

4. Sprint Execution Evidence

4.1 GitHub Collaboration

The project repository was created at the beginning of the sprint with a basic project structure that included a README file, requirements file, and the dataset reference. After the initial setup, a ‘develop’ branch was created to serve as the main integration branch for the sprint.

During development, the team followed a branch-per-feature workflow. Each user story was implemented in a separate feature branch. When the feature was completed, a pull request was opened and reviewed before the changes were merged into the ‘develop’ branch. This approach allowed multiple team members to work in parallel while keeping the main codebase stable.

All team members participated in the pull request process. Feature branches were reviewed before merging, and team members checked whether the code produced the expected results, whether functions were readable and modular, and whether basic documentation such as docstrings was included.

Commit messages followed a consistent naming format:

US-XX: description

This convention helped maintain traceability between code changes and user stories defined during sprint planning.

After all feature branches were reviewed and validated in the ‘develop’ branch, the final integration was merged into main to represent the completed sprint deliverable.

Commit Conventions:

To maintain clear traceability between development work and sprint planning, each commit referenced the related user story identifier. This made it easier to track how individual code changes corresponded to backlog items.

Peer Review:

Every feature PR required at least one reviewer before merging. During review, team members focused on several aspects of code quality:

- Correctness of the output
- Readability and modularity of the functions
- Presence of basic documentation such as docstrings

This review process helped ensure that no feature was merged without verification by another team member.

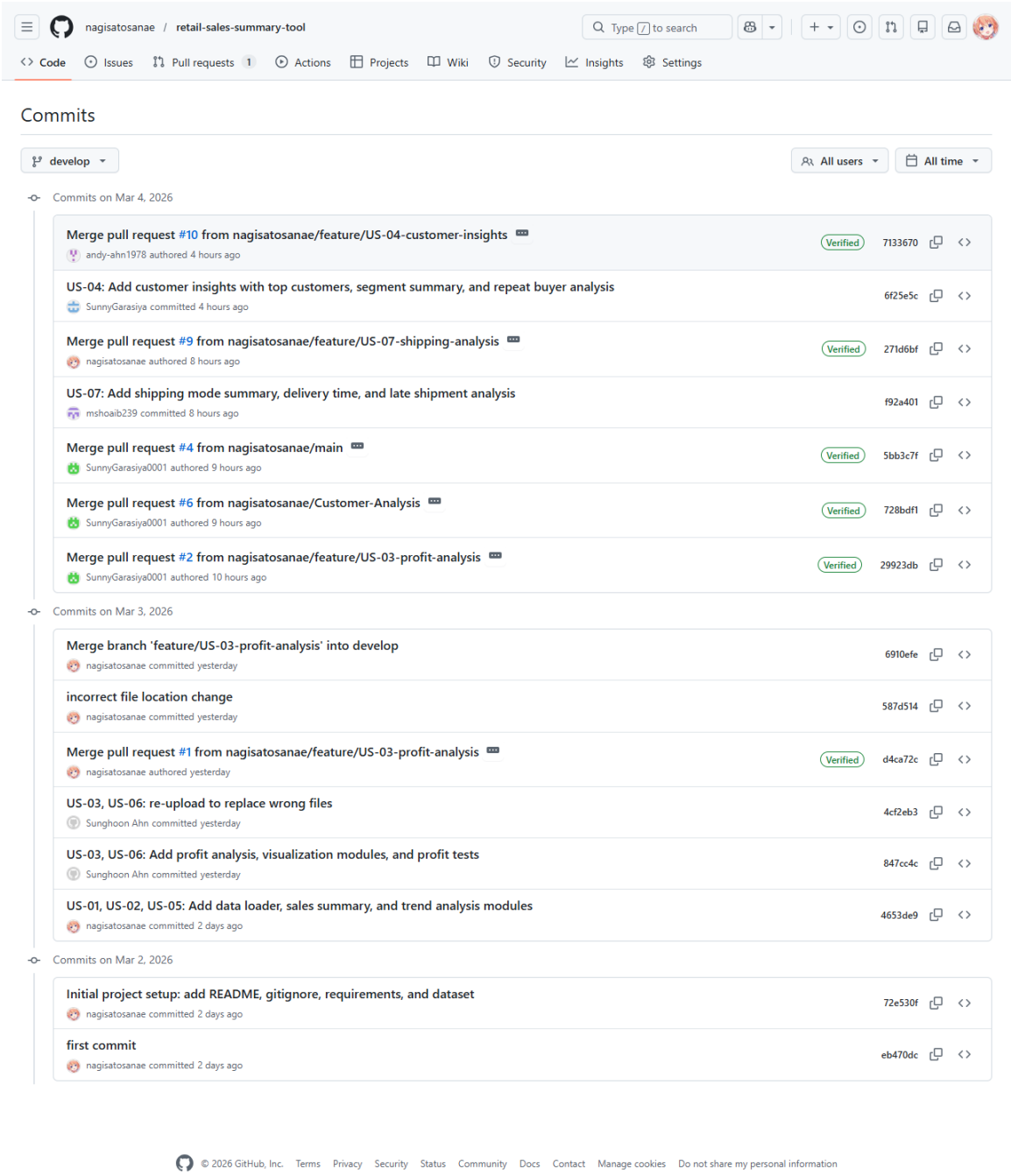

Figure 1. GitHub commit history during the sprint

4.2 Taiga Board

Our Taiga board was organized with the following columns:

- **New** — Backlog items not yet started

- **In Progress** — Currently being worked on
- **Ready for Test** — Code complete and pull request submitted, awaiting review
- **Closed** — Reviewed, merged, and meets Definition of Done

The team used Taiga to track sprint progress and manage tasks during development. All user stories were created in the Taiga backlog at the beginning of the sprint and then broken down into smaller implementation tasks. This made it easier to distribute work among team members and monitor progress throughout the week.

Tasks initially appeared in the 'New' column and were moved to 'In Progress' once a team member started working on them. After implementation was completed and a pull request was submitted, tasks were moved to 'Ready for Test' while waiting for review.

Once the code review was completed and the pull request was approved and merged, the corresponding task was moved to 'Closed'. Updating the board regularly helped the team keep track of which tasks were active, which ones were waiting for review, and which stories had already been completed.

Using the board also made it easier to visualize overall sprint progress and ensured that tasks moved steadily toward completion during the one-week sprint.

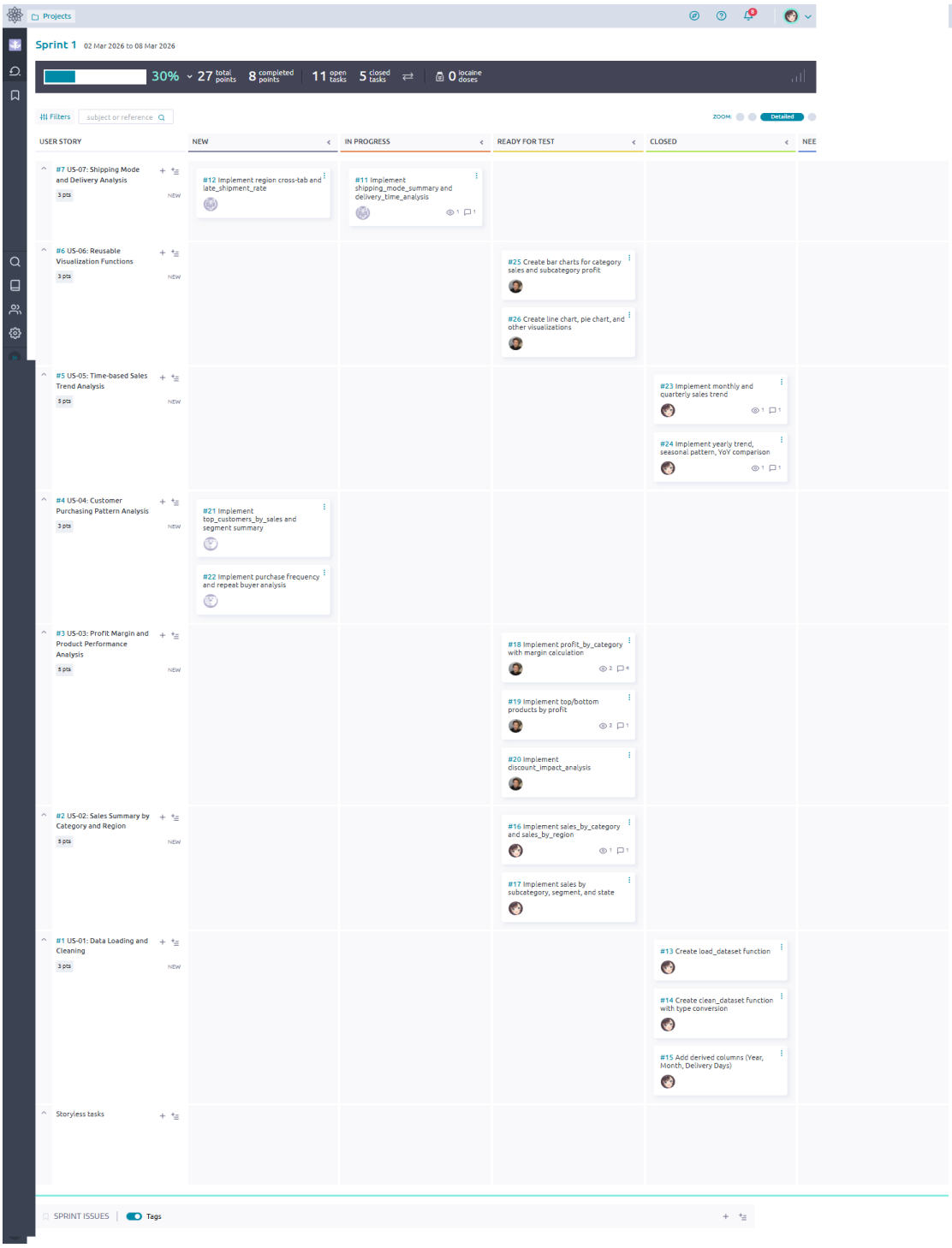

Figure 2. Taiga sprint board showing task progress.

5. Tool Demonstration Summary

The Retail Sales Summary Tool processes the Superstore dataset and generates several analytical summaries related to sales performance, profitability, customer behavior, time-based trends, and shipping patterns. The objective of the tool is not to build a full application, but to provide reusable Python functions that can produce meaningful insights from the dataset.

The analysis begins with a data preparation step, where the dataset is loaded and basic cleaning is performed. The Superstore dataset contains approximately 10,000 rows recorded at the line-item level, meaning that a single order may appear across multiple rows if multiple products were purchased. During preprocessing, the tool standardizes data types, handles missing values when necessary, and creates derived columns that support later analysis.

After the dataset is prepared, the tool generates several types of analytical outputs.

Sales Analysis

The sales analysis module summarizes revenue distribution across product categories, regions, and customer segments. Results show that Technology generates the highest overall sales, followed by Furniture and Office Supplies. Regional comparisons indicate that the West region produces the largest total revenue among the four regions. Segment analysis also shows that the Consumer segment represents the largest share of total sales.

Profit Analysis

Profit analysis focuses on margin performance across categories and sub-categories. The results indicate that Technology products generally achieve higher profit margins, while several Furniture sub-categories perform less strongly. One notable observation is that the Tables sub-category generates negative profit despite relatively high sales, suggesting that discounting may significantly affect profitability.

Customer Insights

ustomer analysis identifies 793 unique customers in the dataset. The results show that repeat buyers tend to generate substantially higher total revenue compared with customers who purchase only once. While the Consumer segment contains the largest number of customers, the Corporate segment shows a higher average spending level per customer.

Sales Trend Analysis

Time-based analysis reveals a consistent growth pattern in sales over the observed years. The results also show a clear seasonal pattern, with sales increasing significantly during the fourth quarter of each year. In particular, November and December consistently represent the highest sales months across categories.

Visualization Functions

To support future visualization or dashboard integration, the project also includes several reusable plotting functions. These functions generate matplotlib.Figure objects and can be used directly by visualization frameworks such as Streamlit or other reporting tools.

Shipping Analysis

The shipping analysis module examines delivery performance across different shipping modes. The results indicate that Standard Class is the most commonly used shipping option, accounting for the majority of orders. Same Day shipping provides the fastest delivery times but is used less frequently. The tool also calculates late shipment rates to help monitor potential logistics delays.

Overall, the tool demonstrates how modular Python functions can be used to transform raw transactional data into structured analytical insights.

6. Sprint Reflection

What Went Well

The sprint progressed steadily, and the team was able to complete all planned user stories within the one-week timeline. One factor that contributed to this outcome was consistent communication among team members. Since not everyone had prior experience with Git-based collaboration, we frequently communicated through Microsoft Teams to ask questions, share screens, and troubleshoot issues together. This helped reduce confusion and allowed the team to resolve technical problems quickly.

Another positive aspect was the way the project was structured through user stories. By defining clear user stories at the beginning of the sprint, each team member had a clear understanding of their responsibilities. The completion of the data loading task early in the sprint was particularly helpful because it provided a stable dataset structure that other analysis modules could build upon. Once this foundation was ready, several analysis tasks could be developed in parallel without major dependencies.

In addition, the use of GitHub and Taiga helped keep the development process organized. pull requests and task tracking made it easier to see what work had already been completed and what tasks were still in progress. This visibility helped the team coordinate work and maintained steady progress throughout the sprint.

Challenges Faced

One of the main challenges during the sprint was the team's initial unfamiliarity with Git workflows. Some members had limited experience with creating ‘branches’, submitting ‘pull requests’, or resolving ‘merge’ conflicts. As a result, the early stages of the sprint required additional time for learning and coordination. However, through collaboration and discussion, team members gradually became more comfortable with the process.

Another challenge involved understanding the structure of the dataset. At first, we interpreted each row in the dataset as representing a full order. Later we realized that the dataset records data at the line-item level, meaning that a single order may appear across multiple rows when multiple products are included in the same purchase. This required us to adjust our analysis logic, particularly when calculating order counts and customer metrics.

Although these issues slowed progress slightly at the beginning, they were resolved early enough that the team could continue development without major delays.

What We Would Change Next Time

If we were to repeat this sprint, several improvements could make the process more efficient.

First, we would begin the sprint with a short exploratory data analysis session. Spending time examining the dataset structure before writing user stories would help the team better understand the data and avoid early misunderstandings.

Second, we would introduce brief daily stand-up meetings. While asynchronous communication worked reasonably well, short synchronous check-ins could help identify blockers earlier and improve coordination among team members.

Finally, writing basic tests earlier in the development process could help verify functionality sooner. In this sprint, most tests were written after implementation. Introducing tests earlier would likely improve code reliability and reduce the need for later adjustments.

How We Applied Agile Principles

Throughout the sprint, the team attempted to follow several core Agile principles. Communication and collaboration were prioritized when problems appeared, particularly when dealing with Git workflow issues or dataset interpretation. Instead of strictly following the original plan, the team discussed issues and adapted the workflow when necessary.

The sprint also focused on delivering working code incrementally. By dividing the project into user stories and implementing them through separate feature 'branches', the team was able to develop and integrate features step by step. This approach helped maintain stability in the codebase while allowing multiple members to work simultaneously.

The sprint provided practical experience in applying Agile practices within a small collaborative project.

Submitted for CPSC-620-3 Agile Software Development — Assignment 2